

SIEM & Alerting Build Report

Wazuh deployment, Suricata network IDS, and real-time Telegram alert pipeline

Author: Edwin Sekgethela Date: 12 May 2026 Status: Complete

1. Overview

This document covers the deployment of the core SIEM stack and the build-out of a real-time alerting pipeline that pushes qualifying Wazuh alerts directly to a Telegram channel, enabling instant analyst notification without needing to keep the dashboard open.

SIEM PLATFORM	IDS ENGINE	ALERT CHANNEL
Wazuh 4.7.3	Suricata 8.0.3	Telegram Bot API

2. Infrastructure

The lab runs on a 2019 MacBook Pro (Intel i7, 16 GB RAM) using VirtualBox. Two dedicated VMs handle separate functions:

VM	IP ADDRESS	RAM	CPU	ROLE
soc-lab	192.168.56.30	6 GB	2 cores	Wazuh Manager + Agent
target-lab	192.168.56.20	4 GB	2 cores	Suricata IDS + Wazuh Agent

3. Build Phases

Phase 1 — Wazuh Stack Deployment

Deployed Wazuh using the official Docker single-node deployment on the soc-lab VM.

```
# Clone the Wazuh Docker repository
git clone https://github.com/wazuh/wazuh-docker.git
cd wazuh-docker/single-node

# Generate SSL certificates
docker compose -f generate-indexer-certs.yml run --rm generator

# Start all containers
docker compose up -d
```

Three containers came up successfully:

- wazuh.manager:4.7.3 — alert processing and integrations
- wazuh.indexer:4.7.3 — event storage (OpenSearch)
- wazuh.dashboard:4.7.3 — web UI on port 443

Phase 2 — Telegram Bot Integration

Created a Telegram bot via @BotFather and built a custom Python integration script inside the Wazuh manager container.

Bot token and Telegram chat ID redacted for this public version. The original bot token was briefly exposed during development and was immediately revoked and rotated via @BotFather — see the Troubleshooting Log, Issue 01, for details. Treat both values as secrets: load them from environment variables or a local config file excluded via .gitignore, never hardcode them in a script that gets committed.

```
# Exec into the manager container
docker exec -it single-node-wazuh.manager-1 /bin/bash

# Write the integration script
cat > /var/ossec/integrations/custom-telegram << 'EOF'
#!/usr/bin/env python3
import sys, json, requests, os

TELEGRAM_TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN")
CHAT_ID = os.environ.get("TELEGRAM_CHAT_ID")

with open(sys.argv[1]) as f:
    alert = json.load(f)

level = alert.get('rule', {}).get('level', 'N/A')
desc = alert.get('rule', {}).get('description', 'N/A')
agent = alert.get('agent', {}).get('name', 'N/A')
ts = alert.get('timestamp', 'N/A')

msg = f'Wazuh Alert\nLevel: {level}\nDesc: {desc}\nAgent: {agent}\nTime: {ts}'

requests.post(f'https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage',
              json={'chat_id': CHAT_ID, 'text': msg, 'parse_mode': 'Markdown'})
EOF

# Set permissions
chmod 750 /var/ossec/integrations/custom-telegram
chown root:wazuh /var/ossec/integrations/custom-telegram

# Install requests library
pip3 install requests
```

Phase 3 — Wazuh Integration Config

Added the integration block to `ossec.conf` and restarted the manager:

```
<integration>
  <name>custom-telegram</name>
  <hook_url>https://api.telegram.org</hook_url>
  <level>3</level>
  <alert_format>json</alert_format>
</integration>

# Restart Wazuh manager
/var/ossec/bin/wazuh-control restart
```

Phase 4 — Wazuh Agent on soc-lab

Installed the matching Wazuh agent (4.7.3) on the soc-lab VM and registered it with the manager:

```
# Add Wazuh repository
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | gpg --dearmor | sudo tee /usr/share/keyrings/wazuh.gpg > /dev/null
echo 'deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/ stable main' | sudo tee
/etc/apt/sources.list.d/wazuh.list

# Install matching version
sudo apt-get update && sudo apt-get install -y wazuh-agent=4.7.3-1

# Register agent with manager
sudo /var/ossec/bin/agent-auth -m 127.0.0.1

# Start agent
sudo systemctl enable wazuh-agent && sudo systemctl start wazuh-agent
```

Phase 5 — Suricata Port Scan Rule

Added a custom Nmap SYN scan detection rule to Suricata on target-lab and updated rules via `suricata-update`:

```
# Update Suricata rules
sudo suricata-update

# Add custom Nmap SYN scan detection rule
sudo tee -a /var/lib/suricata/rules/suricata.rules << 'EOF'
alert tcp any any -> $HOME_NET any (msg:"ET SCAN Nmap SYN Scan Detected";
  flow:stateless; flags:S,12;
  threshold:type threshold,track by_src,count 20,seconds 3;
  classtype:network-scan; sid:9000001; rev:1;)
EOF

# Reload Suricata
sudo systemctl restart suricata
```

4. Verification

The following tests confirmed end-to-end pipeline functionality:

TEST	RESULT	EVIDENCE
Telegram webhook	PASS	Bot responded to <code>/getMe</code> with correct bot name
Manual script test	PASS	<code>python3 custom-telegram /tmp/test-alert.json</code> — message received
SSH brute force	PASS	Rule 5710 level 5 fired — Telegram alert received
Nmap SYN scan	PASS	ET SCAN Nmap SYN Scan Detected — multiple Telegram alerts
Suricata ET INFO	PASS	P2P traffic flagged from target-lab automatically

5. Final Component Status

COMPONENT	STATUS	NOTES
Wazuh Manager	UP	Docker container, port 55000
Wazuh Indexer	UP	Docker container, port 9200
Wazuh Dashboard	UP	Docker container, port 443
Wazuh Agent (soc-lab)	UP	systemd service, version 4.7.3
Wazuh Agent (target-lab)	UP	Shipping Suricata alerts
Suricata IDS	UP	target-lab, version 8.0.3
Telegram Integration	LIVE	Alerts firing in real time